

Writing Our First Spring Boot Application

Goal of this lecture :

- Create our first working Spring Boot application.
- Understand the basic flow.
- Build curiosity for upcoming concepts.

Key Takeaway :

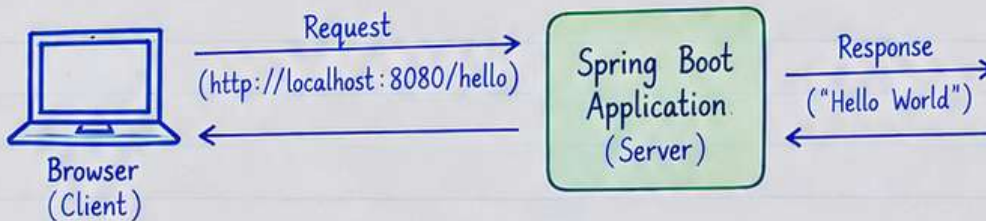
Focus on building a working application first. We will go deep dive step by step in upcoming lectures.

1 What Are We Doing Today ?

- In this lecture, we are going to write our first Spring Boot application.
- The goal is not to understand every internal concept immediately. That is intentional.
- Spring Boot hides a lot of complexity in the beginning so that developers can quickly start building applications.
- In this lecture, we will experience how fast Spring Boot helps us create a working web application, and in later lectures, we will slowly understand what is happening behind the scenes.

2 What Are We Building ?

- We are going to build a simple web application where the browser sends a request to our Spring Boot application, and the application sends back a response.
- The flow will look like this :



We will create one endpoint :

`localhost:8080/hello`

When we open this URL in the browser, it will return :

`Hello World`

3 What is a Port ?

- Suppose your laptop is connected to a network.
- Your laptop has an IP address.
- Example : 192.168.1.10
- Now imagine many applications are running on the same laptop :
 - Chrome
 - WhatsApp
 - Zoom
 - Spotify
 - Spring Boot application
- If some data arrives at your laptop, how will the operating system know which application should receive that data ?
- This is where port numbers come in.

Think :

Your laptop can run many applications at the same time. Ports help the OS to identify which application should receive the incoming data.

4 IP Address vs Port Number

IP Address

- Identifies the machine.
- Example : 192.168.1.10

Port Number

- Identifies the application running inside that machine.
- Example : 192.168.1.10:8080

Meaning :

- Machine = 192.168.1.10
- Application = Listening on port 8080

So the IP address brings the data to the correct machine, and the port number helps the operating system deliver that data to the correct application.

5

What is Localhost ?

- localhost means your own machine.
- It is a special hostname that always points to your local computer.
- It is equivalent to the IP address : `127.0.0.1`
- So when we write : `localhost:8080`
it means : Send the request to my own machine on port 8080.
- If our Spring Boot application is running on port 8080, then the browser can send a request to it using : `http://localhost:8080`

Remember :

- ★ localhost = your own machine
- ★ localhost = 127.0.0.1
- ★ Used for local development
- ★ Does not require internet
- ★ Commonly used with port 8080 in Spring Boot

6

How Does the Browser Know Which Port to Use ?

- When we type a normal website URL like :

`www.coderarmy.in`

we usually do not mention any port number.

- That is because browsers use default ports.
- For example :

Protocol	Default Port
http	80
https	443

- So when we write : `https://www.coderarmy.in`
the browser automatically assumes port 443.
- When we write : `http://www.coderarmy.in`
the browser automatically assumes port 80.
- But when we are running a local Spring Boot application, it usually runs on port 8080, so we mention it manually :

`http://localhost:8080`

Why Different Ports ?

- Different applications use different ports so they can run on the same machine.
- Each application listens on a unique port to receive requests.
- Web servers use standard ports (80, 443) for public websites.
- Spring Boot uses 8080 by default to avoid conflict with these standard ports.

Note :

Port 80 is usually used for HTTP.
Port 443 is usually used for HTTPS.
Port 8080 is commonly used for local development with Spring Boot.

7

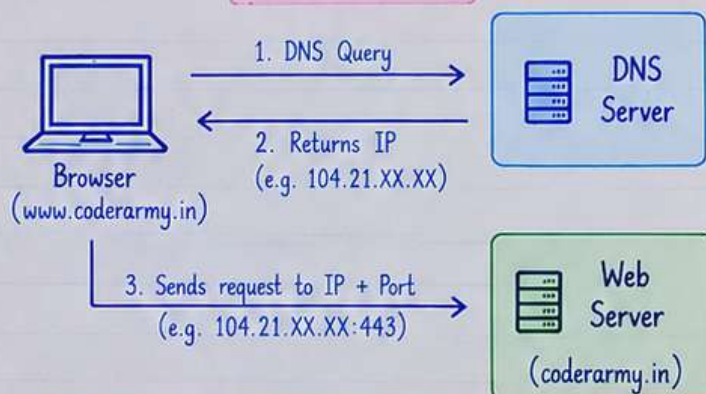
DNS and Port

- When we type a domain name like :
`www.coderarmy.in`
- the browser first uses DNS (Domain Name System) to get the IP address of that domain.
- After getting the IP address, the browser sends the request to the appropriate port.
- The port is either mentioned in the URL or decided by the protocol default.

Key Takeaway :

- ★ DNS gives the IP address of a domain.
- ★ The port is either mentioned in the URL or decided by the protocol default.
- ★ For local development, we use localhost:8080.

How it works ?



Example : `https://www.coderarmy.in`

- DNS resolves `www.coderarmy.in` to an IP address.
- HTTPS tells the browser to use port 443 by default.

8 Creating a Spring Boot Project

Using Spring Initializr

- 1 Open <https://start.spring.io>
- 2 Choose Project : Maven (or Gradle)
- 3 Choose Language : Java
- 4 Choose Spring Boot version : (latest stable)
- 5 Enter Project details :
 - Group : com.example
 - Artifact : demo
 - Name : demo
 - Package name : com.example.demo
- 6 Add Dependencies (for now) :
 - Spring Web
- 7 Click on Generate, download the ZIP file and open it in IntelliJ IDEA.

Project Details (Example)

- Project : Maven
- Language : Java
- Spring Boot : 3.x.x (latest)
- Group : com.example
- Artifact : demo
- Name : demo
- Package name : com.example.demo
- Dependencies : Spring Web

Note :

- ★ Spring Initializr sets up the basic project structure for us.
- ★ It automatically adds required configuration.
- ★ We just need to focus on writing code.

9 Understanding the Generated Project Structure

```
demo/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/demo/
│   │   │       └── DemoApplication.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
└── pom.xml
```

Important Files

File / Folder	Purpose
DemoApplication.java	Main class to start the application.
application.properties	Configuration file (e.g. port number).
pom.xml	Maven configuration file (manages dependencies).
src/main/java	Contains our Java code.
src/main/resources	Contains configuration files and static files.
src/test/java	Contains test cases (we'll see later).

10 Writing Our First Controller

- Create a new class called `HelloController` inside the package `com.example.demo`.
- Use `@RestController` to make it a REST controller.
- Create a method with `@GetMapping("/hello")` that returns a simple string.

HelloController.java

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

Annotations Used

Annotation	Meaning
<code>@RestController</code>	Tells Spring that this class will handle HTTP requests and return data directly (usually JSON or text).
<code>@GetMapping("/hello")</code>	Maps the HTTP GET request for /hello to this method.

What Happens Here ?

- ★ When the browser sends a GET request to /hello
- ★ Spring finds the method with `@GetMapping("/hello")`.
- ★ The method returns the string "Hello World",
- ★ The browser receives and displays the response.

11 Writing Our First Controller

Now we will create a simple REST controller to return "Hello World".

DemoApplication.java

```

1 package com.example.demo;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.
   SpringBootApplication;
5
6
7 @SpringBootApplication
8 public class DemoApplication {
9     public static void main(String[] args) {
10         SpringApplication.run(DemoApplication.class, args);
11     }
12 }

```

Explanation :

- ★ `@SpringBootApplication` → Main annotation that enables Spring Boot.
- ★ `public static void main(...)` → Entry point of the application.
- ★ `SpringApplication.run(...)` → Starts the Spring Boot application.
- ★ When we run this file, an embedded server (Tomcat) will start on port 8080.

12 Creating a REST Controller

Now create a new Java class called `HelloController.java` in the package `com.example.demo`.

HelloController.java

```

1 package com.example.demo;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class HelloController {
8
9     @GetMapping("/hello")
10    public String hello() {
11        return "Hello World";
12    }
13 }

```

Explanation :

- ★ `@RestController` → Marks this class as a REST API controller.
- ★ `@GetMapping("/hello")` → Maps HTTP GET requests to `/hello` URL.
- ★ `hello()` → A simple method that returns a string response.
- ★ When the browser sends a GET request to `/hello`, this method will be executed.

13 Running the Application

- 1 Open `DemoApplication.java`.
- 2 Click on the Run ► button in IntelliJ IDEA.
- 3 Wait for the application to start (you will see logs in the console).
- 4 Open your browser and go to :
`http://localhost:8080/hello`
- 5 You should see: `Hello World`

What Happens Internally ?

- ★ Spring Boot starts an embedded Tomcat server on port 8080.
- ★ It scans for our controller.
- ★ It maps the `/hello` URL to the `hello()` method.
- ★ When we hit the URL, it calls the method and returns the response.

Key Takeaways

- ★ We created our first Spring Boot application.
- ★ We learned the concept of port and localhost.
- ★ We used Spring Initializr to set up the project.
- ★ We created a simple REST controller with `/hello` endpoint.
- ★ We saw how quickly we can build a working web application using Spring Boot.

"A simple Hello World is the first step towards building real-world applications."